```
User.all
# SELECT "users".* FROM "users" WHERE "users"."state" = 'pending'

User.active
# SELECT "users".* FROM "users" WHERE "users"."state" = 'pending' AND
"users"."state" = 'active'

User.where(state: 'inactive')
# SELECT "users".* FROM "users" WHERE "users"."state" = 'pending' AND
"users"."state" = 'inactive'
```

As you can see above the `default_scope` is being merged in both `scope` and `where` conditions.

14.4 Removing All Scoping

If we wish to remove scoping for any reason we can use the `unscoped` method. This is especially useful if a `default_scope` is specified in the model and should not be applied for this particular query.

```
Client.unscoped.load
```

This method removes all scoping and will do a normal query on the table.

Note that chaining `unscoped` with a `scope` does not work. In these cases, it is recommended that you use the block form of `unscoped`:

```
Client.unscoped {
  Client.created_before(Time.zone.now)
}
```

15 Dynamic Finders

For every field (also known as an attribute) you define in your table, Active Record provides a finder method. If you have a field called `first_name` on your `Client` model for example, you get `find_by_first_name` for free from Active Record. If you have a `locked` field on the `Client` model, you also get `find_by_locked` and methods.

You can specify an exclamation point (`!`) on the end of the dynamic finders to get them to raise an `ActiveRecord::RecordNotFound` error if they do not return any records, like `Client.find_by_name!("Ryan")`

If you want to find both by name and locked, you can chain these finders together by simply typing "`and`" between the fields. For example, `Client.find_by_first_name_and_locked("Ryan", true)`.

16 Understanding The Method Chaining

The Active Record pattern implements Method Chaining, which allow us to use multiple Active Record methods together in a simple and straightforward way.

You can chain methods in a statement when the previous method called returns an `ActiveRecord::Relation`, like `all`, `where`, and `joins`. Methods that return a single object (seeRetrieving a Single Object Section) have to be at the end of the statement.

There are some examples below. This guide won't cover all the possibilities, just a few as examples. When an Active Record method is called, the query is not immediately generated and sent to the database, this just happens when the data is actually needed. So each example below generates a single query.

16.1 Retrieving filtered data from multiple tables

```
Person
  .select('people.id, people.name, comments.text')
  .joins(:comments)
  .where('comments.created_at > ?', 1.week.ago)
```